

claude-windows / 54d4074a-5a8d-4058-b42d-a757c338132c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra\54d4074a-5a8d-4058-b42d-a757c338132c\subagents\workflows\wf_1c9ea2fd-eb8\agent-af5a3ff835761ed02.jsonl`  
- SHA-256: `d0e9f932c58fae89f69587c5c700b2220dad769ad253f948c04100b84bbc371e`  
- Source modified: `2026-06-21T19:34:19+00:00`  
- Imported at: `2026-07-05T16:48:22+00:00`  
- project: `wf_1c9ea2fd-eb8`  
- session_id: `54d4074a-5a8d-4058-b42d-a757c338132c`
```

Transcript

1. user (2026-06-21T19:32:10.686Z)

You are reviewing two NEW test files for SOUNDNESS + FLAKINESS. Read under
C:/Users/avidu/Projects/khelsutra-cov:

```
- web/src/components/SelectionFooter.test.tsx  
- web/src/i18n/catalog-invariants.test.ts  
- web/src/components/SelectionFooter.tsx and web/src/i18n/index.ts (to confirm the tests match the real component/contract)  
- web/src/test/setup.ts (the jsdom localStorage shim) and an existing test (web/src/components/a8.status.test.tsx) for the house conventions.
```

Adversarially assess:

1. DETERMINISM: the i18n instance is a shared singleton whose language is global. Do the tests properly set/reset language (beforeEach/afterEach) so they are order-independent and cannot leak kn/hi state into other test files run in the same vitest process? Is there any async race (changeLanguage not awaited, missing act())?
2. TAUTOLOGY: is any assertion always-true regardless of the code (e.g. asserting absence of a string that could never appear anyway)?
3. CORRECTNESS: does `screen.getByRole('button')` really resolve to a single element in every render? Are the en plural/summary assertions ("4 rallies", "Building?") correct against the en catalog?
4. Any brittle exact-string pin that a legitimate future re-wording would break unnecessarily? Return structured findings. Reason statically; do not run the suite.

2. assistant (2026-06-21T19:32:12.953Z)

I'll review these test files adversarially. Let me start by reading all the relevant files.

3. user (2026-06-21T19:32:13.919Z)

```
1      // @vitest-environment jsdom  
2      //  
3      // Coverage + REGRESSION guard for SelectionFooter's floor-warning block (previously  
uncovered) AND  
4      // the kn/hi native-review fixes (#77). The warning renders footer.floorOne / floorAll /  
5      // floorSomeLead+floorSomeRest ? the very keys the adversarial audit cleaned of the  
leaked  
6      // `min_shot_count` token and the "engine"/"server" jargon. Rendering the REAL catalog  
in en/kn/hi and  
7      // asserting the cleaned copy means a regression that re-introduces the token/jargon  
fails right here  
8      // (not just in the data-level catalog-invariants test) ? i.e. the bug can't reach the  
screen.  
9      import { describe, it, expect, beforeEach, afterEach, vi } from "vitest";  
10     import { render, screen, cleanup, fireEvent } from "@testing-library/react";  
11     import i18n from "../i18n";  
12     import { SelectionFooter } from "../SelectionFooter";
```

```

13     import type { RallySegment } from "../api/types";
14
15     // Only the array LENGTH matters to the footer (floorSomeLead shows atRisk.length);
    content is unused.
16     const risks = (n: number): RallySegment[] => Array.from({ length: n }, () => ({} as
    RallySegment));
17     const baseProps = { totalSec: 90, reelName: "", onReelNameChange: () => {}, onBuild: ()
    => {} };
18     const warnText = () => screen.getByTestId("floor-warning").textContent ?? "";
19     const buildBtn = () => screen.getByRole("button") as HTMLButtonElement; // the footer
    has exactly one button
20
21     afterEach(cleanup);
22
23     describe("SelectionFooter ? floor warning renders the audit-clean copy (en)", () => {
24         beforeEach(async () => {
25             window.localStorage.clear();
26             await i18n.changeLanguage("en");
27         });
28
29         it("allDropped + a single selection ? floorOne, passive phrasing (no 'engine'), Build
    blocked", () => {
30             render(<SelectionFooter {...baseProps} count={1} warning={{ minShotCount: 6, atRisk:
    risks(1), allDropped: true }} />);
31             expect(warnText()).toContain("6"); // {min}
32             expect(warnText()).toMatch(/skipped/i);
33             expect(warnText()).not.toMatch(/engine/i); // audit: en is engine-free, so kn/hi/en
    stay engine-free
34             expect(buildBtn().disabled).toBe(true); // every selected rally below the floor ?
    build would fail
35         });
36
37         it("allDropped + many ? floorAll (count + min), Build blocked", () => {
38             render(<SelectionFooter {...baseProps} count={3} warning={{ minShotCount: 6, atRisk:
    risks(3), allDropped: true }} />);
39             expect(warnText()).toContain("3"); // {count}
40             expect(warnText()).toContain("6"); // {min}
41             expect(warnText()).not.toMatch(/engine/i);
42             expect(buildBtn().disabled).toBe(true);
43         });
44
45         it("partial risk ? floorSomeLead + floorSomeRest, NO leaked min_shot_count token,
    Build allowed", () => {
46             render(<SelectionFooter {...baseProps} count={5} warning={{ minShotCount: 6, atRisk:
    risks(2), allDropped: false }} />);
47             const t = warnText();
48             expect(t).toContain("2"); // floorSomeLead count = atRisk.length
49             expect(t).toContain("6"); // {min}
50             expect(t).not.toMatch(/min_shot_count/); // audit: the internal code token must
    never surface
51             expect(t).not.toMatch(/engine/i);
52             expect(buildBtn().disabled).toBe(false); // only SOME are at risk ? build can
    proceed
53         });
54     });
55
56     describe("SelectionFooter ? floor warning is jargon-free in kn + hi (#77 regression)",
    () => {
57         beforeEach(() => window.localStorage.clear());
58         afterEach(async () => {
59             await i18n.changeLanguage("en");
60         });
61
62         it("kn: floorOne + floorSome carry no '?????' and no leaked token", async () => {
63             await i18n.changeLanguage("kn");

```

```

64     render(<SelectionFooter {...baseProps} count={1} warning={{ minShotCount: 6, atRisk:
risks(1), allDropped: true }} />);
65     expect(warnText()).not.toContain("?????");
66     cleanup();
67     render(<SelectionFooter {...baseProps} count={5} warning={{ minShotCount: 6, atRisk:
risks(2), allDropped: false }} />);
68     expect(warnText()).not.toContain("?????");
69     expect(warnText()).not.toMatch(/min_shot_count/);
70   });
71
72   it("hi: floorOne + floorSome carry no '????' and no leaked token", async () => {
73     await i18n.changeLanguage("hi");
74     render(<SelectionFooter {...baseProps} count={1} warning={{ minShotCount: 6, atRisk:
risks(1), allDropped: true }} />);
75     expect(warnText()).not.toContain("????");
76     cleanup();
77     render(<SelectionFooter {...baseProps} count={5} warning={{ minShotCount: 6, atRisk:
risks(2), allDropped: false }} />);
78     expect(warnText()).not.toContain("????");
79     expect(warnText()).not.toMatch(/min_shot_count/);
80   });
81 });
82
83 describe("SelectionFooter ? no-warning controls (summary, reel name, build button)", ()
=> {
84   beforeEach(async () => {
85     window.localStorage.clear();
86     await i18n.changeLanguage("en");
87   });
88
89   it("no warning ? no floor-warning region; summary + enabled Build render", () => {
90     render(<SelectionFooter {...baseProps} count={4} warning={null} />);
91     expect(screen.queryByTestId("floor-warning")).toBeNull();
92     expect(screen.getByTestId("selection-summary").textContent).toMatch(/4 rallies/);
93     expect(buildBtn().disabled).toBe(false);
94   });
95
96   it("building ? the button shows the 'Building?' label and is disabled", () => {
97     render(<SelectionFooter {...baseProps} count={2} warning={null} building />);
98     expect(buildBtn().textContent).toMatch(/Building/);
99     expect(buildBtn().disabled).toBe(true);
100   });
101
102   it("empty selection ? Build disabled", () => {
103     render(<SelectionFooter {...baseProps} count={0} warning={null} />);
104     expect(buildBtn().disabled).toBe(true);
105   });
106
107   it("editing the reel name and clicking Build fire their callbacks", () => {
108     const onReelNameChange = vi.fn();
109     const onBuild = vi.fn();
110     render(<SelectionFooter {...baseProps} count={3} warning={null}
onReelNameChange={onReelNameChange} onBuild={onBuild} />);
111     fireEvent.change(screen.getByLabelText("Reel name"), { target: { value: "finals" }
});
112     expect(onReelNameChange).toHaveBeenCalledWith("finals");
113     fireEvent.click(buildBtn());
114     expect(onBuild).toHaveBeenCalled();
115   });
116 });
117

```

4. user (2026-06-21T19:32:14.436Z)

```

1    // Content regression guards for the localization catalogs ? the rules the adversarial

```

```

translation
2      // audit (#77) established, pinned so the specific bugs can never silently return. This
complements
3      // hygiene.test.ts (which proves key/placeholder PARITY) by asserting catalog CONTENT:
4      // 1. no internal code-token / <code> leaks in any locale,
5      // 2. the brand "KhelSutra" is never transliterated into an Indic script,
6      // 3. engine/server/bucket jargon appears ONLY on keys where the en source uses it,
7      // 4. the specific flagged mistranslations stay fixed.
8      // These are pure data assertions over repo-root /locales (D-CATALOG-HOME) ? no render
context needed.
9      import { describe, it, expect } from "vitest";
10     import { readFileSync, readdirSync } from "node:fs";
11     import { join } from "node:path";
12
13     const LOCALES_DIR = join(import.meta.dirname, "..", "..", "..", "locales");
14
15     type Flat = Record<string, string>;
16     function flatEntries(obj: Record<string, unknown>, prefix = ""): Flat {
17         const out: Flat = {};
18         for (const [k, v] of Object.entries(obj)) {
19             const key = prefix ? `${prefix}.${k}` : k;
20             if (v && typeof v === "object" && !Array.isArray(v)) Object.assign(out,
flatEntries(v as Record<string, unknown>, key));
21             else out[key] = String(v);
22         }
23         return out;
24     }
25     const load = (loc: string): Flat =>
26         flatEntries(JSON.parse(readFileSync(join(LOCALES_DIR, loc, "common.json"), "utf8"))) as
Record<string, unknown>;
27
28     const ALL_LOCALES = readdirSync(LOCALES_DIR, { withFileTypes: true })
29         .filter((d) => d.isDirectory())
30         .map((d) => d.name);
31     const en = load("en");
32
33     describe("catalog content ? no internal token / HTML-tag leaks (every locale)", () => {
34         for (const loc of ALL_LOCALES) {
35             it(`${loc}: no 'min_shot_count' code token or <code> tag in any user-facing string`,
() => {
36                 const offenders = Object.entries(load(loc))
37                     .filter(([, v]) => /min_shot_count/.test(v) || /\<\/?code>/i.test(v))
38                     .map(([k]) => k);
39                 expect(offenders).toEqual([]);
40             });
41         }
42     });
43
44     describe("catalog content ? the brand is never transliterated (Indic locales)", () => {
45         const TRANSLIT: Record<string, string> = { kn: "????????", hi: "????????" };
46         for (const loc of ["kn", "hi"] as const) {
47             it(`${loc}: no Indic transliteration of "KhelSutra" anywhere`, () => {
48                 const offenders = Object.entries(load(loc))
49                     .filter(([, v]) => v.includes(TRANSLIT[loc]))
50                     .map(([k]) => k);
51                 expect(offenders).toEqual([]);
52             });
53             it(`${loc}: the ingest console keys keep the Latin brand`, () => {
54                 const e = load(loc);
55                 for (const key of ["ingest.intro", "ingest.reassure", "ingest.fileHint"]) {
56                     expect(e[key], `${loc} ${key}`).toContain("KhelSutra");
57                 }
58             });
59         }
60     });

```

```

61
62     describe("catalog content ? engine/server/bucket jargon only where en uses it (Indic
locales)", () => {
63         // en deliberately avoids these terms in user-facing body copy (it says "KheISutra" /
"cloud-storage"
64         // / passive voice) ? EXCEPT the technical uriAriaLabel ("engine host", "bucket URL").
So the rule is
65         // self-maintaining: a localized jargon term is allowed ONLY on a key whose en value
also uses the
66         // English word. Reintroducing "engine"/"server"/"bucket" anywhere else fails here.
67         const JARGON: Record<string, ReadonlyArray<readonly [string, string]>> = {
68             kn: [
69                 ["?????", "engine"],
70                 ["?????", "server"],
71                 ["?????", "bucket"],
72             ],
73             hi: [
74                 ["?????", "engine"],
75                 ["?????", "server"],
76                 ["?????", "bucket"],
77             ],
78         };
79         for (const loc of ["kn", "hi"] as const) {
80             it(`${loc}: a jargon term appears only on keys whose en value also uses it`, () => {
81                 const e = load(loc);
82                 const violations: Array<{ key: string; term: string; en: string }> = [];
83                 for (const [key, val] of Object.entries(e)) {
84                     for (const [term, eng] of JARGON[loc]) {
85                         if (val.includes(term) && !(en[key] ?? "").toLowerCase().includes(eng)) {
86                             violations.push({ key, term, en: en[key] ?? "(absent)" });
87                         }
88                     }
89                 }
90                 expect(violations).toEqual([]);
91             });
92         }
93     });
94
95     describe("catalog content ? flagged mistranslations stay fixed (Kannada/Hindi word
choice)", () => {
96         it("kn bulk.invert is not the 'rotate' mistranslation (????????)", () => {
97             expect(load("kn")["bulk.invert"]).not.toBe("????????");
98         });
99         it("kn result.dismiss is not the legalistic '????????'", () => {
100             expect(load("kn")["result.dismiss"]).not.toBe("????????");
101         });
102         it("hi result.dismiss is not the harsh '????? ?????'", () => {
103             expect(load("hi")["result.dismiss"]).not.toBe("????? ?????");
104         });
105     });
106

```

5. user (2026-06-21T19:32:14.910Z)

```

1     // Sticky compile footer (PER_RALLY_SELECTION.md P4): live selection summary, a reel-
name input,
2     // the Build button, and ? above all ? an HONEST min_shot_count warning. The engine
silently drops
3     // selected rallies below stitching.min_shot_count; we surface that BEFORE Build so a
rally never
4     // vanishes without a reason (D7). We warn, we do NOT override the floor (D11).
5     import { useTranslation, Trans } from "react-i18next";
6     import { mmss } from "../lib/format";
7     import type { FloorRisk } from "../lib/floor";
8

```

```

9      interface Props {
10         count: number;
11         totalSec: number;
12         reelName: string;
13         onReelNameChange: (v: string) => void;
14         onBuild: () => void;
15         /** A compile is in flight (Build wired in P5). */
16         building?: boolean;
17         /** Non-null when some selected rallies sit below the engine floor. */
18         warning: FloorRisk | null;
19     }
20
21     export function SelectionFooter({ count, totalSec, reelName, onReelNameChange, onBuild,
building, warning }: Props) {
22         const { t } = useTranslation();
23         const buildDisabled = count === 0 || building || warning?.allDropped === true;
24
25         return (
26             <footer className="fixed inset-x-0 bottom-0 border-t border-black/10 bg-white/95
backdrop-blur">
27                 {warning && (
28                     <div
29                         role="status"
30                         data-testid="floor-warning"
31                         className={
32                             "border-b px-6 py-2 text-sm " +
33                             (warning.allDropped
34                                 ? "border-red-200 bg-red-50 text-red-800"
35                                 : "border-amber-200 bg-amber-50 text-amber-900")
36                         }
37                     >
38                         {warning.allDropped ? (
39                             count === 1 ? (
40                                 <Trans i18nKey="footer.floorOne" values={{ min: warning.minShotCount }}
components={{ b: <strong /> }} />
41                             ) : (
42                                 <Trans i18nKey="footer.floorAll" values={{ count, min:
warning.minShotCount }} components={{ b: <strong /> }} />
43                             )
44                             ) : (
45                                 <>
46                                     <strong>{t("footer.floorSomeLead", { count: warning.atRisk.length
})}</strong>{" "}
47                                     <Trans i18nKey="footer.floorSomeRest" values={{ min: warning.minShotCount
}} components={{ code: <code /> }} />
48                                 </>
49                             )}
50                                 </div>
51                             )}
52
53                             <div className="mx-auto flex max-w-5xl flex-wrap items-center gap-3 px-6 py-3">
54                                 <span className="text-sm text-ink/80" data-testid="selection-summary">
55                                     <strong className="text-ink">{t("footer.count", { count })}</strong>
56                                     {t("footer.summaryRest", { total: mmss(totalSec) })}
57                                 </span>
58
59                                 <label className="ml-auto flex items-center gap-2 text-sm text-ink/60">
60                                     <span className="hidden sm:inline">{t("footer.reelName")}</span>
61                                     <input
62                                         type="text"
63                                         value={reelName}
64                                         onChange={(e) => onReelNameChange(e.target.value)}
65                                         placeholder={t("footer.reelNamePlaceholder")}
66                                         aria-label={t("footer.reelName")}
67                                         className="min-h-11 w-40 rounded-lg border border-black/15 bg-white px-3

```

```

py-1.5 text-ink outline-none focus:border-green"
67         />
68     </label>
69
70     <button
71         type="button"
72         onClick={onBuild}
73         disabled={buildDisabled}
74         className="min-h-11 rounded-full bg-green px-5 py-2 text-sm font-semibold
text-[#04140D] disabled:opacity-40"
75     >
76         {building ? t("footer.building") : t("footer.build", { count })}
77     </button>
78 </div>
79 </footer>
80 );
81 }
82

```

6. user (2026-06-21T19:32:15.852Z)

```

1    // i18n scaffold (tracker B1) ? the shared localization foundation for KhelSutra Studio.
2    //
3    // Per docs/FRIENDLY_AND_MULTILINGUAL.md + the engine's I18N_PLAN.md:
4    // - keys, never prose (t("app.title"), enforced later by a no-bare-strings lint, B2);
5    // - ICU MessageFormat for interpolation/plurals (Kannada/Hindi plural rules live in
the catalog,
6    //     not the code ? replaces the SPA's hand-rolled `count > 1 ? "s" : ""`);
7    // - one negotiation order shared with the marketing site (B3);
8    // - always-fallback to `en` so a missing translation degrades to English, never a raw
key.
9    import i18n from "i18next";
10   import { initReactI18next } from "react-i18next";
11   import LanguageDetector from "i18next-browser-languagedetector";
12   import ICU from "i18next-icu";
13   // Catalogs are HOISTED to repo-root /locales (D-CATALOG-HOME) ? the single source of
truth shared
14   // by this SPA and the marketing site's t() shim (tracker B3). Vite resolves the
workspace root via
15   // the repo `.git` marker, so importing the parent-dir /locales works in dev, build, and
viteest.
16   import enCommon from "../../locales/en/common.json";
17   import knCommon from "../../locales/kn/common.json";
18   import hiCommon from "../../locales/hi/common.json";
19   import esCommon from "../../locales/es/common.json";
20   import daCommon from "../../locales/da/common.json";
21   import idCommon from "../../locales/id/common.json";
22
23   // Supported locales: en/kn/hi foundation (FRIENDLY D6) + MULTILINGUAL_EXPANSION Phase A
(es/da/id,
24   // Latin, catalog-only). Anything else falls back to `en`.
25   export const SUPPORTED_LOCALES = ["en", "kn", "hi", "es", "da", "id"] as const;
26   export type Locale = (typeof SUPPORTED_LOCALES)[number];
27   export const DEFAULT_LOCALE: Locale = "en";
28
29   // ONE persisted-preference key, shared with the marketing site's t() shim (D9 / tracker
B3), so a
30   // visitor's language choice carries across both surfaces.
31   export const LANG_STORAGE_KEY = "khelsutra.lang";
32
33   // Catalogs live in repo-root /locales (D-CATALOG-HOME), consumed by both surfaces. `kn`
+ `hi`
34   // shipped (B5/B6, AI-drafted ? native review pending). A missing key in any locale
still falls back
35   // to `en` ? the by-design invariant.

```

```

36     const resources = {
37       en: { common: enCommon },
38       kn: { common: knCommon },
39       hi: { common: hiCommon },
40       es: { common: esCommon },
41       da: { common: daCommon },
42       id: { common: idCommon },
43     } as const;
44
45     void i18n
46       .use(ICU)
47       .use(LanguageDetector)
48       .use(initReactI18next)
49       .init({
50         resources,
51         fallbackLng: DEFAULT_LOCALE,
52         supportedLngs: [...SUPPORTED_LOCALES],
53         nonExplicitSupportedLngs: true, // en-IN -> en
54         defaultNS: "common",
55         ns: ["common"],
56         interpolation: { escapeValue: false }, // React already escapes
57         returnNull: false,
58         // Negotiation order (I18N_PLAN §2.3): persisted pref -> ?lang= -> browser -> en.
59         detection: {
60           order: ["localStorage", "querystring", "navigator"],
61           lookupLocalStorage: LANG_STORAGE_KEY,
62           lookupQuerystring: "lang",
63           caches: ["localStorage"],
64         },
65         react: { useSuspense: false },
66       });
67
68     // Keep <html lang> in sync with the active locale (screen readers + SEO use it).
69     Reflects the
70     // resolved content language, so an `en` fallback for an absent locale still reads as
71     `en`.
72     function syncDocumentLang(lng: string) {
73       if (typeof document !== "undefined") {
74         document.documentElement.lang = (i18n.resolvedLanguage || lng ||
75         DEFAULT_LOCALE).split("-")[0];
76       }
77     }
78     i18n.on("languageChanged", syncDocumentLang);
79     syncDocumentLang(i18n.language ?? DEFAULT_LOCALE);
80
81     export default i18n;

```

7. user (2026-06-21T19:32:16.321Z)

```

1      // Vitest setup (runs before each test file's imports). DOM-env only: jsdom under Node
26 does not
2      // expose a Web Storage `localStorage`, which the i18n language detector and the
3      persistence test
4      // both need. Node-env test files (typeof window === "undefined") skip this entirely.
5      if (typeof window !== "undefined" && typeof globalThis.localStorage === "undefined") {
6        let store: Record<string, string> = {};
7        const mem: Storage = {
8          get length() {
9            return Object.keys(store).length;
10          },
11          clear() {
12            store = {};
13          },
14          getItem(key: string) {

```



```

14         return Object.prototype.hasOwnProperty.call(store, key) ? store[key] : null;
15     },
16     key(index: number) {
17         return Object.keys(store)[index] ?? null;
18     },
19     removeItem(key: string) {
20         delete store[key];
21     },
22     setItem(key: string, value: string) {
23         store[key] = String(value);
24     },
25 };
26 Object.defineProperty(window, "localStorage", { value: mem, configurable: true,
writable: true });
27 Object.defineProperty(globalThis, "localStorage", { value: mem, configurable: true,
writable: true });
28 }
29

```

8. user (2026-06-21T19:32:16.662Z)

```

1     // @vitest-environment jsdom
2     //
3     // A8 ? honest expectations for non-tech users on slow uplinks: background-processing
reassurance and
4     // status shown as icon+TEXT chips (never icon-only), on BOTH the ingest progress and
the build state.
5     import { describe, it, expect, beforeEach, afterEach, vi } from "vitest";
6     import { render, screen, cleanup, fireEvent, act } from "@testing-library/react";
7     import i18n from "../i18n";
8     import { CompileResult } from "../CompileResult";
9     import { IngestPanel } from "../IngestPanel";
10    import type { JobStatus } from "../api/types";
11
12    beforeEach(async () => {
13        window.localStorage.clear();
14        await i18n.changeLanguage("en");
15    });
16
17    afterEach(() => {
18        cleanup();
19        vi.restoreAllMocks();
20        vi.unstubAllGlobals();
21        vi.useRealTimers();
22    });
23
24    describe("A8: build state shows status + reassurance (CompileResult)", () => {
25        it("building ? a 'stitching?' chip + a reassurance line (and nothing to dismiss yet)",
() => {
26            render(<CompileResult build={{ status: "building" }} onDismiss={() => {}} />);
27            expect(screen.getByText("Stitching your reel?")).toBeTruthy();
28            expect(screen.getByText(/only takes a moment/i)).toBeTruthy();
29            expect(screen.queryByRole("button", { name: "Dismiss" })).toBeNull();
30        });
31
32        it("done ? a ? icon paired with the 'ready' text (icon+text, not icon-only)", () => {
33            render(<CompileResult build={{ status: "done", url: "/x", filename: "r.mp4" }}
onDismiss={() => {}} />);
34            const h = screen.getByRole("heading", { level: 2 });
35            expect(h.textContent).toContain("?");
36            expect(h.textContent).toContain("Your reel is ready");
37        });
38
39        it("error ? a ? icon paired with 'failed' + the message", () => {
40            render(<CompileResult build={{ status: "error", error: "boom" }} onDismiss={() =>

```

```

{}} />);
41     const h = screen.getByRole("heading", { level: 2 });
42     expect(h.textContent).toContain("?");
43     expect(screen.getByText("boom")).toBeTruthy();
44   });
45 });
46
47 describe("A8: ingest progress reassurance + status chips (IngestPanel)", () => {
48   const httpOk = (s: number, j: unknown) => ({ ok: s >= 200 && s < 300, status: s, json:
async () => j, text: async () => JSON.stringify(j) });
49   function fetchSeq(process: unknown, jobs: JobStatus[]) {
50     let i = 0;
51     return vi.fn(async (url: string | URL, init?: RequestInit) => {
52       const u = String(url);
53       if (u === "/api/process" && (init?.method ?? "GET") === "POST") return process;
54       if (u.startsWith("/api/jobs/")) {
55         const b = jobs[Math.min(i, jobs.length - 1)];
56         i += 1;
57         return httpOk(200, b);
58       }
59       throw new Error(`unexpected fetch ${u}`);
60     }) as unknown as typeof fetch;
61   }
62
63   it("while processing ? the working chip + 'safe to wait' reassurance", async () => {
64     vi.useFakeTimers();
65     vi.stubGlobal("fetch", fetchSeq(httpOk(202, { job_id: "j", status: "queued" }), [
66       { job_id: "j", status: "running", progress: "segmenting" },
67     ]));
68     render(<IngestPanel onLoaded={vi.fn()} />);
69     fireEvent.change(screen.getByLabelText(/video location/i), { target: { value:
"gs://b/x.mp4" } });
70     fireEvent.click(screen.getByRole("button", { name: /process video/i }));
71     await act(async () => {
72       await vi.advanceTimersByTimeAsync(0); // submit ? polling
73     });
74     await act(async () => {
75       await vi.advanceTimersByTimeAsync(1000); // running tick ? progress word
76     });
77     expect(screen.getByText(/processing your video/i)).toBeTruthy();
78     expect(screen.getByText(/safe to wait/i)).toBeTruthy();
79   });
80
81   it("on failure ? a ? 'Couldn't process' status chip alongside the friendly message",
async () => {
82     vi.stubGlobal("fetch", vi.fn(async () => {
83       throw new Error("offline");
84     }) as unknown as typeof fetch);
85     render(<IngestPanel onLoaded={vi.fn()} />);
86     fireEvent.change(screen.getByLabelText(/video location/i), { target: { value:
"gs://b/x.mp4" } });
87     fireEvent.click(screen.getByRole("button", { name: /process video/i }));
88     expect(await screen.findByText("Couldn't process")).toBeTruthy();
89     expect(await screen.findByText(/is it running/i)).toBeTruthy();
90   });
91 });
92

```

9. assistant (2026-06-21T19:32:20.567Z)

Now let me look at the actual en/kn/hi catalog content for the keys under test to verify the assertions match reality.

10. user (2026-06-21T19:32:23.654Z)

```

==== en ====
'floorOne' => '<b>Your only selected rally</b> has fewer than {min} shots ? it would be skipped,
so the build would fail. Pick a longer rally.'
'floorAll' => "<b>All {count} selected rallies</b> have fewer than {min} shots ? they'd all be
skipped, so the build would fail. Pick longer rallies."
'floorSomeLead' => '{count, plural, one {# rally} other {# rallies}}'
'floorSomeRest' => "with fewer than {min} shots will be skipped when building (they're too
short).\"
'count' => '{count, plural, one {# rally} other {# rallies}}'
'summaryRest' => 'selected . {total} total'
'build' => 'Build reel ({count})'
'building' => 'Building?'
'reelName' => 'Reel name'
'reelNamePlaceholder' => 'my-highlights'
==== kn ====
'floorOne' => '<b>????? ???? ?????????? ??????</b>????? {min} ?????? ?????? ????\u200c????? ?
????? ?????? ?????? ??????, ?????? ???? ?????? ??????. ?????? ?????? ?????? ??????.'
'floorAll' => '<b>????? ?????? {count} ??????</b> {min} ?????? ?????? ????\u200c????? ?
????? ?????? ?????? ??????, ?????? ???? ?????? ??????. ?????? ?????? ?????? ??????.'
'floorSomeLead' => '{count, plural, one {# ??????} other {# ??????}}'
'floorSomeRest' => '{min} ?????? ?????? ????\u200c????? ?????? ?????? ?????? (??? ?????
?????)'
'count' => '{count, plural, one {# ??????} other {# ??????}}'
'summaryRest' => '????? . ????? {total}'
'build' => '???? ????? ({count})'
'building' => '?????????'
'reelName' => '???? ?????'
'reelNamePlaceholder' => 'my-highlights'
==== hi ====
'floorOne' => '<b>????? ?????? ?????? ????</b> ??? {min} ?? ?? ?????? ??? ? ??? ?????? ??? ?????,
????? ?????? ??? ?????? ?? ?????? ?????? ??????'
'floorAll' => '<b>??? {count} ??????</b> ??? {min} ?? ?? ?????? ??? ? ?? ??? ?????? ??
?????, ?????? ?????? ??? ?????? ?????? ?????? ??????'
'floorSomeLead' => '{count, plural, one {# ??????} other {# ??????}}'
'floorSomeRest' => '????? {min} ?? ?? ?????? ???, ?????? ?????? ??? ?????? ?? ?????? (?? ?????? ???
???)?'
'count' => '{count, plural, one {# ??????} other {# ??????}}'
'summaryRest' => '????? . ??? {total}'
'build' => '??? ?????? ({count})'
'building' => '?? ??? ???'
'reelName' => '??? ?? ???'
'reelNamePlaceholder' => 'my-highlights'

```

11. assistant (2026-06-21T19:32:26.979Z)

Now let me verify a couple of important details: the en assertions ("4 rallies", "skipped"), and check whether any locale's floor copy could contain the jargon transliterations, and whether the ``` tag handling in `floorSomeRest` matters. Let me also confirm the test setup wiring (vitest config / setup registration) and the `FloorRisk` type.`

12. user (2026-06-21T19:32:27.938Z)

```

1      // The stitching.min_shot_count floor ? surfaced, never hidden (PER_RALLY_SELECTION.md
D7/D11).
2      // /api/compile silently drops selected rallies below the floor; the UI WARNS before
Build so a
3      // rally never vanishes without explanation. An override is deferred (owner gate, Launch
Report).
4      import type { RallySegment } from "../api/types";
5      import type { Selection } from "../selection";
6      import { selectedRallies } from "../selection";
7
8      export interface FloorRisk {
9          /** The configured stitching.min_shot_count the engine enforces. */
10         minShotCount: number;

```

```

11      /** Selected rallies the engine WILL drop at compile (numeric shot_count below the
floor). */
12      atRisk: RallySegment[];
13      /** True when EVERY selected rally is below the floor ? /api/compile returns 400
(build fails). */
14      allDropped: boolean;
15    }
16
17    /** Which SELECTED rallies the engine's min_shot_count floor would drop at compile.
18    * Mirrors backend/main.py:362-383 EXACTLY: a rally is dropped iff it has a NUMERIC
shot_count
19    * strictly below the floor; a rally WITHOUT a shot_count is EXEMPT (kept). Returns null
when there
20    * is nothing to warn about (no floor configured, empty selection, or none at risk). */
21    export function floorRisk(
22      rallies: RallySegment[],
23      selection: Selection,
24      minShotCount: number | undefined,
25    ): FloorRisk | null {
26      if (minShotCount == null) return null;
27      const selected = selectedRallies(rallies, selection);
28      if (selected.length === 0) return null;
29      const atRisk = selected.filter((r) => {
30        const sc = r.metadata?.shot_count;
31        return typeof sc === "number" && sc < minShotCount; // exempt when shot_count is
absent
32      });
33      if (atRisk.length === 0) return null;
34      return { minShotCount, atRisk, allDropped: atRisk.length === selected.length };
35    }
36

```

13. user (2026-06-21T19:32:29.474Z)

```

---VITE---
import { defineConfig } from "vitest/config";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

// The SPA makes SAME-ORIGIN relative `/api/*` calls. In the appliance (prod) a reverse
// proxy serves the SPA and proxies `/api` to the engine on 127.0.0.1:8000
// (docs/LOCAL_APPLIANCE_ARCHITECTURE.md). In dev, Vite proxies `/api` to the local engine
// (`python -m backend.main --server`, :8000). Override with VITE_ENGINE_ORIGIN if needed.
export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: {
    proxy: {
      "/api": { target: "http://localhost:8000", changeOrigin: true },
    },
  },
  test: {
    // Default env is node (the pure lib/parser tests); the i18n render test opts into jsdom via
a
    // `@vitest-environment jsdom` docblock. Match .test.tsx too so render tests are
discovered.
    environment: "node",
    include: ["src/**/*.test.{ts,tsx}"],
    // DOM-env localStorage polyfill (jsdom under Node 26 ships none); node-env files no-op.
    setupFiles: ["src/test/setup.ts"],
    // Coverage floor (CI runs `npm run test:coverage`): a merge can't DROP coverage below these
// thresholds, so untested new code is caught. `all` counts every source file (not only the
ones a
    // test imported), so deleting a test surfaces here too. Excludes are non-logic files: the
app
    // entry/bootstrap, type-only declarations, and the test harness itself.

```

```

coverage: {
  provider: "v8",
  reporter: ["text", "text-summary"],
  all: true,
  include: ["src/**/*.{ts,tsx}"],
  exclude: [
    "src/**/*.test.{ts,tsx}", // the tests
    "src/test/**", // jsdom localStorage shim (test harness, not shipped logic)
    "src/main.tsx", // app entry: mounts <App/> + bundles fonts ? nothing to unit-test
    "src/api/types.ts", // type-only declarations (no runtime code)
    "**/*.d.ts",
  ],
  // Floor set just below current (stmts/lines 92.6, branches 87.7, funcs 82.4) ? a real
  ratchet
  // that fails CI on a meaningful drop, with a little slack so it doesn't flap. Ratchet
  upward as
  // coverage grows. (functions is kept looser: App.tsx is the engine-orchestrating shell,
  costly
  // to exercise fully in unit tests.)
  thresholds: {
    statements: 92,
    branches: 87,
    functions: 80,
    lines: 92,
  },
},
},
});

```

14. assistant (2026-06-21T19:32:36.694Z)

Now I have a strong picture. Let me verify a few specific adversarial concerns:

1. Does the en `floorSomeRest` contain a ``` tag? It does NOT (it's plain text). The component wraps `components={{ code: <code /> }}` but the string has no `` markup ? fine.`
2. Is `getByRole("button")` truly a single button? The footer renders exactly one ``. The text input is `type="text"` (not a button). Good.
3. The kn/hi tests assert absence of jargon transliterations on floor copy ? let me confirm none of the floor strings accidentally contain those terms (which would make the assertion non-tautological and meaningful).

Let me check whether the jargon terms appear anywhere in kn/hi floor keys, and check the negative-assertion tautology concern carefully.

15. user (2026-06-21T19:32:42.391Z)

```

==== kn term= ?????? ====
floorOne -> contains term? False
floorAll -> contains term? False
floorSomeLead -> contains term? False
floorSomeRest -> contains term? False
  term anywhere in catalog? True
  min_shot_count anywhere? False
==== hi term= ???? ====
floorOne -> contains term? False
floorAll -> contains term? False
floorSomeLead -> contains term? False
floorSomeRest -> contains term? False
  term anywhere in catalog? True
  min_shot_count anywhere? False

```

16. assistant (2026-06-21T19:32:46.037Z)

Important finding. The jargon terms (????? / ????) appear in the catalog but NOT in any floor

key, and `min\_shot\_count` appears nowhere in any catalog. This is the heart of the tautology concern. Let me dig into where the jargon term DOES appear (the `uriAriaLabel` "engine host" exemption) to assess the render-test value.

17. user (2026-06-21T19:32:48.775Z)

```
==== kn keys containing ?????? ====
  ingest.uriAriaLabel => ?????? ???? ? gs:// ????? URL ???? ?????? ?????????????? ??
==== hi keys containing ???? ====
  ingest.uriAriaLabel => ?????? ?? ????? ? ?? gs:// ???? URL ?? ????? ???? ? ? ? ? ?
```

18. assistant (2026-06-21T19:32:59.609Z)

This confirms the picture. The jargon term only appears in `ingest.uriAriaLabel`, which `SelectionFooter` never renders. So the kn/hi `not.toContain("?????")` assertions in `SelectionFooter.test.tsx` are tautological at the render level ? the term physically cannot appear in the footer's floor warning because it's not in any footer key. Let me verify the English assertions one more time and check the `count===1` floorSomeLead detail and `**` rendering, then finalize.**

The en assertions to verify:

- "4 rallies" ? `count: {{ plural one {# rally} other {# rallies}}}` ? count=4 ? "4 rallies" ? (summary uses `footer.count`)
- "Building?" ? `footer.building` = "Building?" ? (test matches `/Building/`)
- floorOne contains "skipped" ?, contains "6" (min) ?, no "engine" ?
- floorAll contains "3" and "6" ?
- floorSomeLead with atRisk.length=2 ? "2 rallies" ? contains "2" ?; floorSomeRest contains "6" ?

Now the critical tautology analysis on the `**` / Trans rendering: floorOne/floorAll use `**` with `components={{ b: <strong /> }}`. The catalog uses `**...</b>` markup. The `textContent` will include the inner text regardless. The "6" appears as `{min}` interpolation ? that's real. Good.******

One more: the en test `not.toMatch(/engine/i)` ? does any en footer floor string contain "engine"? No (verified: en floorOne says "skipped"/"build would fail", no "engine"). So this negative is also tautological for en, but the test comment explicitly acknowledges "en is engine-free". Let me confirm there's no en footer key with "engine".

19. user (2026-06-21T19:33:03.651Z)

```
en footer keys containing engine/server/bucket:
any en key with min_shot_count: []
locales dir: ['da', 'en', 'es', 'hi', 'id', 'kn']
```

20. assistant (2026-06-21T19:33:15.554Z)

All confirmed. The en footer has no engine/server/bucket jargon, and `min\_shot\_count` appears in no locale. Now let me assess the determinism/order-independence concern more carefully across the two describe blocks in `SelectionFooter.test.tsx`.

Key determinism analysis:

- Block 1 (en): `beforeEach` awaits `changeLanguage("en")`. No `afterEach` reset, but next block re-sets.
- Block 2 (kn/hi): `beforeEach` only clears localStorage (does NOT set language). `afterEach` awaits `changeLanguage("en")`. Within each test it awaits `changeLanguage("kn"/"hi")`. The `afterEach` resets to en, so it won't leak to other files. Good ? but note the `beforeEach` doesn't reset language, relying solely on `afterEach`. If `afterEach` runs (it always does), state is restored. This is sound for leak-prevention.
- Block 3 (en): `beforeEach` awaits en.

The cross-file leak risk: if block 2's `afterEach` failed to run, kn/hi could leak. But vitest always runs afterEach. The bigger subtle issue: block 2 has no `afterEach(cleanup)` reset of language in `beforeEach` ? but there IS a top-level `afterEach(cleanup)` at line 21 PLUS the

block-local `afterEach`. Vitest runs both. Order: block-local afterEach (changeLanguage en) and top-level afterEach(cleanup). Both run. Fine.

Now the `act()` concern: `changeLanguage` is awaited (async resolved before render). After render, no further language change happens mid-render, so no act() warning expected. The component uses `react: { useSuspense: false }`. `changeLanguage` before render is the standard safe pattern (matches a8.status.test.tsx which uses the same). Good.

Now let me also verify the `getByRole("button")` single-element claim holds in the `building` state and all states ? yes, footer always renders exactly one ``, and the input is not a button. Solid.

Let me record my findings.

21. user (2026-06-21T19:34:04.121Z)

Structured output provided successfully

22. assistant (2026-06-21T19:34:19.210Z)

I've completed the adversarial review of both test files. Summary of what I found:

****DETERMINISM (clean):**** The i18n singleton's global language is correctly managed. Every block awaits `changeLanguage` and resets to `en` (Block 2's `afterEach`), so kn/hi state cannot leak into sibling test files. `changeLanguage` is always awaited before render, and since language is set before rendering (not mid-render), no `act()` is needed ? matching the house pattern in `a8.status.test.tsx`. `catalog-invariants.test.ts` is fully hermetic (reads JSON from disk, never touches the singleton).

****TAUTOLOGY (the main problem):**** Three sets of render-level negative assertions in `SelectionFooter.test.tsx` are always-true regardless of the code:

- The kn/hi `not.toContain("?????"/"????")` assertions (lines 65,68,76,78) ? that token lives only in `ingest.uriAriaLabel`, a key the footer never renders. It is physically impossible for the footer to show it. ****Major**** ? the docblock claims "the bug can't reach the screen," but the screen can never show it by construction.
- The `not.toMatch(/min\_shot\_count/)` render checks ? the token exists in zero catalogs across all six locales. The real guard correctly lives in `catalog-invariants.test.ts` (which loops `ALL\_LOCALES`).
- The en `not.toMatch(/engine/i)` checks ? no en footer key contains "engine" (the test comment is self-aware about this).

****CORRECTNESS (clean):**** `getByRole('button')` resolves to a single element in every state (the reel-name field is a textbox, not a button). All en pins verified against the catalog: "4 rallies", "Building?", floorOne "skipped"+min, floorAll count+min, floorSomeLead "2".

****BRITTLINESS:**** The mistranslation pins use `not.toBe` (robust ? a legitimate re-wording still passes). One weak spot: `floorSomeLead` is pinned via `toContain('2')`, which doesn't actually verify the ICU plural "other" arm was selected ? suggested tightening to `/2 rallies/`.

Net: the data-level `catalog-invariants.test.ts` carries the real regression value; the render test's negative assertions are largely confidence theater and would be stronger as positive localized-substring checks. Verdict: ****ship-with-fixes****.